

TP3 : Collections (Depuis JDK 21)

Créer les classes suivantes :

1. La classe **Personne** avec :
 - 1.1. Attributs : nom (String), prenom (String) et age (int)
 - 1.2. Constructeur, getters, setters, et toString()
2. La classe **Etudiant** qui hérite de **Personne**, avec :
 - 2.1. Attribut supplémentaire : note (float)
 - 2.2. Constructeur, getters, setters, et toString()
3. La classe **Test1** avec la méthode main pour tester une **ArrayList<Etudiant>**:
 - 3.1. Créer les 4 étudiants suivants :

```
Etudiant e1=new Etudiant("Karimi2", "Karim", 21, 15.5f);  
Etudiant e2=new Etudiant("Karimi1", "Karim", 20, 15);  
Etudiant e3=new Etudiant("Karimi3", "Karim", 21, 10);  
Etudiant e4=new Etudiant("Karimi4", "Karim", 19, 11);
```

- 3.2. Créer une ArrayList<Etudiant> et y ajouter les 4 étudiants, un doublon et une valeur null.
 - 3.3. Parcourir tous les éléments avec **la boucle for** et afficher.
 - 3.4. Ajouter un étudiant au début (addFirst).
 - 3.5. Récupérer l'étudiant à l'**index 2**.
 - 3.6. Récupérer le premier et le dernier étudiant.
 - 3.7. Mettre à jour le **troisième étudiant** avec un nouveau.
 - 3.8. Supprimer le **troisième étudiant**.
 - 3.9. Supprimer le premier et le dernier étudiant.
 - 3.10. Vérifier si un étudiant est présent dans la liste (contains).
 - 3.11. Trier la liste par ordre alphabétique de nom (Collections.sort) ;
4. La classe **Test2** avec la méthode main pour tester une **LinkedList<Etudiant>**:
 - 4.1. Créer les 4 étudiants de la question 3.9.
 - 4.2. Créer une LinkedList<Etudiant> et y ajouter les 4 étudiants, un doublon et une valeur null.
 - 4.3. Refaire les mêmes questions de 3.3 à 3.8 pour la LinkedList. Y-a-t-il des différences ?
 - 4.4. Parcourir la liste avec un Iterator.
5. La classe **Test3** avec la méthode main pour tester un **HashSet<Etudiant>**:
 - 5.1. Créer les 4 étudiants de la question 3.1.
 - 5.2. Créer un HashSet<Etudiant> et y ajouter les 4 étudiants, un doublon et 2 valeur null.

- 5.3. Afficher tous les éléments. Quelle méthode vous avez utilisé pour l’affichage et qu’est-ce-que vous remarquez ?
- 5.4. Afficher la **taille**.
- 5.5. Supprimer un étudiant spécifique.
- 5.6. Vérifier la présence d’un étudiant.
6. La classe **Test4** avec la méthode main pour tester un **TreeSet<Etudiant>**:
 - 6.1. Créer les 4 étudiants de la question 3.1.
 - 6.2. Créer un TreeSet<Etudiant> et y ajouter les 4 étudiants, un doublon et 2 valeur null. Qu’est-ce-que vous remarquez ?
 - 6.3. Dans la classe Etudiant, implémenter l’interface Comparable<Etudiant> pour le tri par note.
 - 6.4. Afficher tous les éléments en utilisant la méthode forEach. Observer le tri automatique par note.
 - 6.5. Supprimer les étudiants qui ont une note supérieur à 15.
 - 6.6. Ajouter un étudiant ayant la même note qu’un autre et observer le comportement.
 - 6.7. Récupérer et afficher le plus petit et le plus grand étudiant (first(), last()).
7. La classe **Test5** avec la méthode main pour tester un **HashMap<String, Etudiant>**:
 - 7.1. Créer les 4 étudiants de la question 3.1.
 - 7.2. Créer un HashMap<String, Etudiant> où la clé est le nom de l’étudiant et la valeur l’objet Etudiant.
 - 7.3. Ajouter les 4 étudiants à la HashMap.
 - 7.4. Récupérer un étudiant par son **nom**.
 - 7.5. Vérifier si une **clé** (nom) ou une **valeur** (Etudiant) est présente.
 - 7.6. Supprimer un étudiant de la map par son **nom**.
 - 7.7. Parcourir toutes les **entrées** (Map.Entry<String, Etudiant>) et les afficher.
 - 7.8. Parcourir toutes les **clés** et toutes les **valeurs** séparément.